

UART Communication and Data Integrity

Module	ET4047
Date	27/09/2026
Lab No.	Lab 4
Author	Luke Griffin

1 Overview

UART (Universal Asynchronous Receiver/Transmitter) is the simplest and most widely used way for two chips to talk: GPS modules, Bluetooth modules, GSM modems and the USB link to your PC all use it. Data travels one bit at a time down a single wire in each direction, with no shared clock. Both sides simply agree on the speed (the *baud rate*) in advance.

In this lab you will send data from an Arduino back to itself, then between two Arduinos, and finally add a **SHA-1 hash** to every message so the receiver can detect whether anything was corrupted on the way.

Learning outcomes

By the end of this lab you should be able to:

- Describe a UART frame (start bit, data bits, stop bit) and calculate transmission time from the baud rate.
- Wire a loopback test and a two-board TX/RX link with a common ground.
- Explain how non-blocking `millis()` timing and a line-buffered receiver work, and why they are needed.
- Use a hash to check message integrity, and explain what a hash can and cannot protect against.

Equipment

Item	Qty	Notes
Arduino Nano (classic) or Nano Every, on Uno-style breakout shield	2	All the code works on either board. Task 1 detects the board automatically.
16x2 LCD with I2C backpack	1-2	I2C address 0x20 on the lab displays.
Jumper wires (male-male / male-female)	~10	
USB cables	2	One per board.
Push button	1	Task 3 only.

Software

- Arduino IDE. Select the board you have: *Tools* → *Board* → *Arduino AVR Boards* → *Arduino Nano (classic)*, or *Arduino megaAVR Boards* → *Arduino Nano Every*

(install the megaAVR package from the Boards Manager first). Classic Nano clones may also need *Tools → Processor → ATmega328P (Old Bootloader)*.

- **LiquidCrystal I2C** library (by Frank de Brabander): *Tools → Manage Libraries...* and search for it.
- **SoftwareSerial** is built in. There is nothing to install.
- **sha1_helper.h** for Task 3: the full listing is in Appendix A (also in the Resources folder).

LCD wiring (all tasks)

I2C LCD connections

From		To	Purpose
LCD GND	→	Nano GND	Ground
LCD VCC	→	Nano 5V	Power
LCD SDA	→	Nano A4	I2C data
LCD SCL	→	Nano A5	I2C clock

LCD wiring (every task)

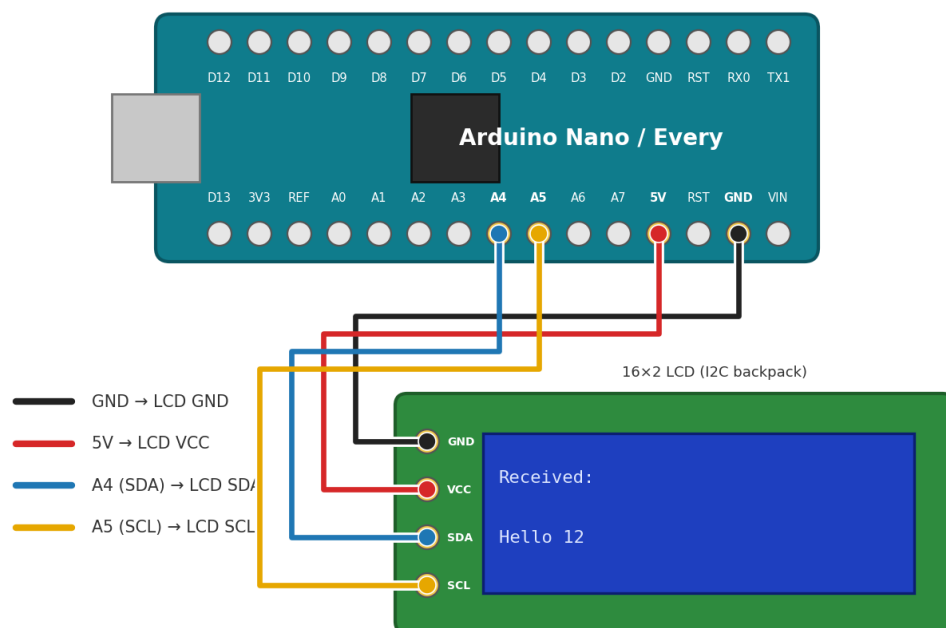


Figure 1: LCD wiring. The pinout is the same on the classic Nano and the Nano Every.

2 How UART works

When the line is idle, the transmitter holds it **high (1)**. To send a byte it:

1. Pulls the line **low** for one bit time. This is the **start bit**, and it tells the receiver “a byte is coming, start your timer now”.
2. Sends the 8 **data bits**, least-significant bit first.

- Returns the line high for one bit time. This is the **stop bit**, which marks the end of the frame.

One UART frame: sending 'A' (0x41 = 0100 0001), least-significant bit first

idle	Start	D0	D1	D2	D3	D4	D5	D6	D7	Stop	idle
1	0	1	0	0	0	0	0	1	0	1	1

Time → each box lasts $1/9600 \text{ s} \approx 104 \mu\text{s}$ at 9600 baud, so one 10-bit frame takes about **1.04 ms**.

Because there is no clock wire, the receiver samples the line at the times it *expects* each bit to arrive. That only works if:

- **Both sides use the same baud rate.** At the wrong rate the receiver samples the wrong bits and you see garbage.
- **TX goes to RX.** One board's transmit pin must drive the other board's receive pin, so the wires cross over.
- **The grounds are connected.** A "high" or "low" voltage only means something relative to a shared 0 V reference.

Hardware UART vs SoftwareSerial

	Hardware UART (Serial / Serial1)	SoftwareSerial
Pins	Fixed: D0 (RX), D1 (TX)	Almost any pins, here D10 (RX) and D11 (TX)
How it works	Dedicated UART peripheral inside the chip	The CPU times each bit in software
Send and receive at once	Yes (full duplex)	No. It cannot receive while it is transmitting.
Reliable speeds	Up to 1 Mbaud+	About 57600 baud at most
Used in	Task 1 (loopback)	Tasks 2 and 3 (board to board)

Why two kinds?

The loopback test **must** use the hardware UART: a SoftwareSerial port cannot listen to itself while it is sending. For the two-board link, SoftwareSerial keeps the hardware pins free and the USB **Serial** Monitor available for debugging.

Writing efficient code for a small microcontroller

The classic Nano has only **2 KB of RAM** and **32 KB of flash**. The code in this lab uses a few habits that keep it small, fast and reliable. Look out for them as you read the listings:

Technique	Example	Why
<code>F()</code> macro for fixed text	<code>Serial.println(F("Sender ready"));</code>	Keeps the text in flash. Without <code>F()</code> , every string literal is also copied into RAM at start-up.

Technique	Example	Why
Smallest suitable type	<code>uint8_t len;</code> instead of <code>int len;</code>	1 byte instead of 2, and faster on an 8-bit CPU. Use it for values that stay between 0 and 255.
Let the compiler count	<code>sizeof(messages) / sizeof(messages[0])</code>	Add a message to the array and the count updates itself. There is no hand-typed 3 to forget.
Drift-free timing	<code>lastSend += SEND_INTERVAL;</code>	<code>lastSend = millis();</code> slowly drifts, because the check happens a little late each time. Adding the interval keeps an exact period.
Fixed buffers, not <code>String</code>	<code>char buffer[MAX_LEN + 1];</code>	The Arduino <code>String</code> class allocates memory at run time, which fragments the heap and can crash a long-running sketch.
Overwrite the LCD, don't clear it	<code>lcdPrintLine(1, msg);</code>	<code>lcd.clear()</code> takes about 2 ms and makes the screen flicker. Writing a full 16-character line, padded with spaces, does neither.

3 Task 1: UART loopback on a single board

Objective: connect the board's UART output straight back into its own input. Everything the board transmits should arrive back on its receive pin, which proves that the UART, the wiring and your code are working before you involve a second board.

Wiring

Loopback connection (plus the LCD wiring from Section 1)

From		To	Purpose
Nano D1 (TX)	→	Nano D0 (RX)	Loopback: output feeds straight into input

Task 1: UART loopback

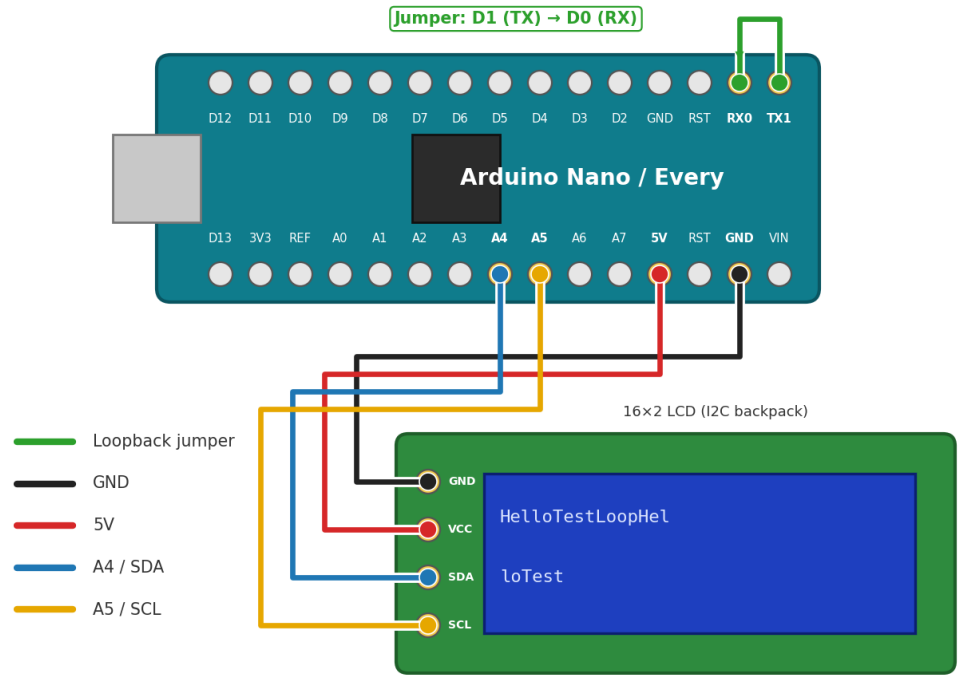


Figure 2: Task 1 wiring: one jumper from D1 (TX) to D0 (RX), which are next to each other on the header

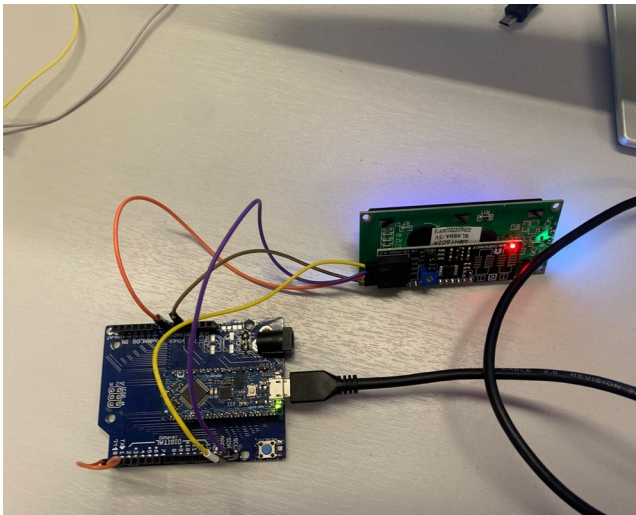


Figure 3: Loopback jumper and LCD wiring



Figure 4: LCD without line wrapping

	Classic Nano (ATmega328P)	Nano Every (ATmega4809)
Hardware UARTs on the pins	One, <code>Serial</code> . It is shared with the USB link.	A spare one, <code>Serial1</code> . USB has its own channel.
Port used by the Task 1 code	<code>Serial</code>	<code>Serial1</code>
Loopback jumper while uploading	Remove it , or the upload fails	Can stay in place

	Classic Nano (ATmega328P)	Nano Every (ATmega4809)
Serial Monitor during Task 1	Shows the messages being sent. Don't type in it: typed characters also arrive on D0.	Stays empty

Why the classic Nano behaves differently

On the classic Nano, D0 and D1 are also wired to the USB-to-serial chip. Anything the board sends on D1 goes both to the jumper and to your PC, and the upload uses the same pins, so the jumper gets in the way while uploading.

The code

Task1_Loopback.ino

```
// Task 1: UART loopback with 16-character line wrapping
// Wiring: jumper from D1 (TX) to D0 (RX)
// Works on both the classic Arduino Nano and the Nano Every.
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

// Choose the hardware UART on D0/D1 for this board:
//   Nano Every -> Serial1 (USB uses a separate channel)
//   Classic Nano -> Serial (shared with USB: remove the jumper while uploading)
#if defined(ARDUINO_AVR_NANO_EVERY) || defined(HAVE_HWSERIAL1)
    #define LINK Serial1
#else
    #define LINK Serial
#endif

const uint8_t LCD_COLS = 16;
const uint8_t LCD_ROWS = 2;
LiquidCrystal_I2C lcd(0x20, LCD_COLS, LCD_ROWS);

const char* const messages[] = {"Hello", "Test", "Loop"};
const uint8_t NUM_MESSAGES = sizeof(messages) / sizeof(messages[0]);

const unsigned long SEND_INTERVAL = 1000; // ms

uint8_t col = 0; // current LCD column
uint8_t row = 0; // current LCD row

// Print one character, wrapping to the next row after 16 characters
void lcdPutChar(char c)
{
    if (col == LCD_COLS)
    {
        col = 0;
        row++;

        if (row == LCD_ROWS) // both rows full: start again at the top
        {
            row = 0;
            lcd.clear();
        }
        lcd.setCursor(col, row);
    }

    lcd.write(c);
    col++;
}
```

```
void setup()
{
    LINK.begin(9600);           // hardware UART on D0 (RX) / D1 (TX)

    lcd.init();
    lcd.backlight();
    lcd.print(F("UART Loopback")); // F() keeps the text in flash, not RAM
    delay(1000);
    lcd.clear();
}

void loop()
{
    static unsigned long lastSend = 0;
    static uint8_t msgIndex = 0;

    // Send the next message once per second (non-blocking, no drift)
    if (millis() - lastSend >= SEND_INTERVAL)
    {
        lastSend += SEND_INTERVAL;
        LINK.print(messages[msgIndex]);
        msgIndex = (msgIndex + 1) % NUM_MESSAGES;
    }

    // Display every byte that has come back in on RX
    while (LINK.available())
    {
        char c = LINK.read();
        if (c == '\r' || c == '\n')
        {
            continue;
        }
        lcdPutChar(c);
    }
}
```

How it works

- **Choosing the port.** The `#if` block runs at compile time. The Nano Every's board package defines `ARDUINO_AVR_NANO_EVERY`, so `LINK` becomes `Serial1`; on a classic Nano it becomes `Serial`. After that, every `LINK` in the code uses the right port, with no other changes needed.
- `LiquidCrystal_I2C lcd(0x20, LCD_COLS, LCD_ROWS)` creates the display object. `0x20` is the backpack's I2C address. If your screen stays blank, run an I2C scanner sketch to find the real address.
- `const char* const messages[]` is an array of strings; both the pointers and the text are read-only. `NUM_MESSAGES` is worked out by the compiler, and `msgIndex = (msgIndex + 1) % NUM_MESSAGES` steps through the messages and wraps back to 0: the same modulo trick as last week's circular queue.
- `LINK.begin(9600)` starts the hardware UART at 9600 baud. Anything printed to `LINK` leaves on D1 and, through the jumper, arrives on D0.
- **The `millis()` timer.** `millis()` returns milliseconds since power-up. Checking `millis() - lastSend >= SEND_INTERVAL` sends once a second **without** `delay()`, so `loop()` keeps running and never misses incoming bytes. `lastSend += SEND_INTERVAL` keeps the period exact, and the subtraction still works when `millis()` wraps round after about 49 days. `static` variables keep their value between calls to `loop()`.

- **The receive loop.** `LINK.available()` returns how many bytes are waiting in the receive buffer; `LINK.read()` takes one out. Line endings are skipped and every other byte goes to `lcdPutChar()`.
- `lcdPutChar()` **wraps the text.** It remembers the cursor position in `col` and `row`. When a row is full it moves to the start of the next row; when both rows are full it clears the screen and starts again at the top-left. It checks for a full row *before* printing, so a character is never written off the edge of the display.

Expected result: the display fills row by row with `HelloTestLoopHel` / `loTestLoopHelloT`, clears, and starts again at the top.

Task 1 exercises

1a Run it

Upload the code, wire the jumper and confirm the messages appear. **(record)** Take a photo of the LCD.

1b Break the loop

While the sketch is running, pull the D1-D0 jumper out, wait a few seconds, then put it back. What happens on the LCD, and why? Is anything lost while the wire is out?

Your answer:

1c Without the wrapping

In the receive loop, temporarily replace `lcdPutChar(c);` with `lcd.write(c);` and run it again. Describe what you see (compare with Figure 4), then put `lcdPutChar()` back. Why does the LCD not wrap on its own?

Your answer:

1d Timing

Using the frame diagram in Section 2, how long does it take to transmit the word `Hello` at 9600 baud? How many characters per second can a 9600-baud link carry at most?

Your answer:

4 Task 2: Communication between two boards

Objective: Board A (the sender) transmits a numbered message every second; Board B (the receiver) collects the characters into a complete line and shows it on its LCD and Serial Monitor.

Wiring

Label your boards **A** (sender) and **B** (receiver). The LCD connects to **Board B** only.

Board-to-board connections (the TX/RX wires cross over)

From		To	Purpose
A D11 (TX)	→	B D10 (RX)	A's data travels to B
A D10 (RX)	→	B D11 (TX)	Return path, used in the extension
A GND	→	B GND	Common reference voltage. Essential.

Task 2: two boards, TX/RX crossed over

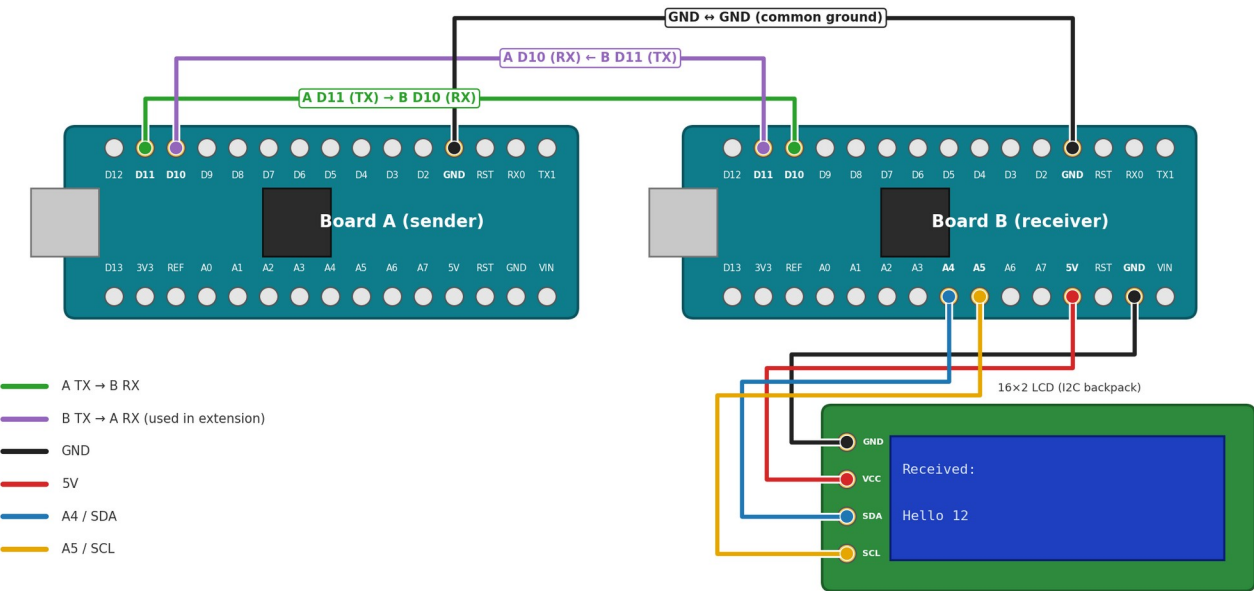


Figure 5: Task 2 wiring: TX goes to RX in both directions, grounds joined, LCD on Board B

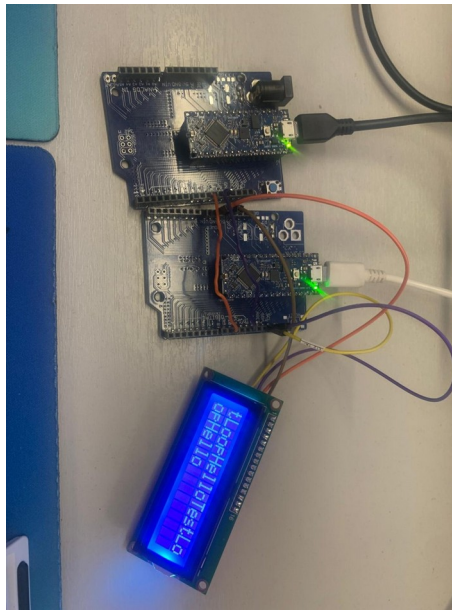


Figure 6: The two-board set-up in the lab

Connect both boards to the PC by USB. Each gets its own COM port. Upload the sender code to A and the receiver code to B, selecting the correct port each time. Open a Serial Monitor for each board, both at **9600 baud**. (In the IDE, open a second window with *File* → *New* to get a second monitor.)

Board A: sender

Task2_Sender.ino

```
// Task 2: Board A (sender)
// Wiring: A D11 (TX) -> B D10 (RX), A D10 (RX) <- B D11 (TX), GND -> GND
#include <SoftwareSerial.h>

SoftwareSerial link(10, 11);          // RX = D10, TX = D11

const char* const messages[] = {"Hello", "Test", "Loop"};
const uint8_t NUM_MESSAGES = sizeof(messages) / sizeof(messages[0]);

const unsigned long SEND_INTERVAL = 1000;  // ms between messages

unsigned long lastSend = 0;
uint8_t msgIndex = 0;
unsigned int counter = 0;

void setup()
{
    link.begin(9600);                  // link to Board B
    Serial.begin(9600);                // USB Serial Monitor for debugging
    Serial.println(F("Sender ready"));
}

void loop()
{
    if (millis() - lastSend >= SEND_INTERVAL)
    {
        lastSend += SEND_INTERVAL;    // fixed period, no drift

        // Frame: text, space, counter, newline e.g. "Hello 12\n"
        link.print(messages[msgIndex]);
        link.print(' ');
        link.print(counter);
        link.print('\n');

        Serial.print(F("Sent: "));
        Serial.print(messages[msgIndex]);
        Serial.print(' ');
        Serial.println(counter);

        msgIndex = (msgIndex + 1) % NUM_MESSAGES;
        counter++;
    }
}
```

- `SoftwareSerial link(10, 11)` creates a second serial port on D10 (RX) and D11 (TX). It behaves exactly like `Serial`: `link.begin()`, `link.print()`, `link.available()`, `link.read()`.
- Every second it sends one **frame**: the message text, a space, a counter and a newline, for example `Hello 12\n`. The newline marks the end of the frame, so the receiver knows where one message stops and the next begins.
- The same text goes to the USB `Serial` Monitor, so you can compare what A sent with what B received.
- Printing the frame with four separate `print()` calls is fine. At 9600 baud the wire is far slower than the CPU, so building the whole string first would not make it faster; it would only use more RAM.

The receiver problem: bytes arrive one at a time

UART delivers **individual bytes**, not whole messages. When A sends **Hello 12**, B's **read()** returns **H**, then **e**, then **l**... and there is no guarantee they all arrive in the same pass of **loop()**. The standard solution is a **line buffer**. The receiver:

1. stores each incoming character in a **char** array and increases an index;
2. when it reads **'\n'**, writes **'\0'** after the last character, turning the array into a normal C string;
3. processes the string, then resets the index to 0 ready for the next message;
4. **never** writes past the end of the array. If a message is too long, it stops storing characters, because writing past the end corrupts whatever variable sits next in memory.

Board B: receiver

Task2_Receiver.ino

```
// Task 2: Board B (receiver with LCD)
#include <SoftwareSerial.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

SoftwareSerial link(10, 11);           // RX = D10, TX = D11

const uint8_t LCD_COLS = 16;
LiquidCrystal_I2C lcd(0x20, LCD_COLS, 2);

const uint8_t MAX_LEN = 32;           // longest message we accept
char buffer[MAX_LEN + 1];             // +1 for the '\0' terminator
uint8_t len = 0;                      // characters stored so far

// Write text on one LCD row and blank the rest of that row.
// Faster and flicker-free compared with lcd.clear() + print.
void lcdPrintLine(uint8_t row, const char* text)
{
    lcd.setCursor(0, row);
    uint8_t i = 0;
    for (; i < LCD_COLS && text[i] != '\0'; i++)
    {
        lcd.write(text[i]);
    }
    for (; i < LCD_COLS; i++)
    {
        lcd.write(' ');
    }
}

// Show one complete message on the Serial Monitor and the LCD
void showMessage(const char* msg)
{
    Serial.print(F("Received: "));
    Serial.println(msg);

    lcdPrintLine(0, "Received:");
    lcdPrintLine(1, msg);
}

void setup()
{
    link.begin(9600);
    Serial.begin(9600);
}
```

```
    lcd.init();  
    lcd.backlight();  
    lcdPrintLine(0, "Waiting...");  
}  
  
void loop()  
{  
    while (link.available())  
    {  
        char c = link.read();  
  
        if (c == '\r')  
        {  
            continue;           // ignore carriage returns  
        }  
  
        if (c == '\n')           // end of message  
        {  
            buffer[len] = '\0';  // turn the buffer into a C string  
            showMessage(buffer);  
            len = 0;             // get ready for the next message  
        }  
        else if (len < MAX_LEN)  // store it if there is room  
        {  
            buffer[len++] = c;  
        }  
    }  
}
```

- `char buffer[MAX_LEN + 1]` has one extra byte for the `'\0'` terminator. `len` counts how many characters are stored so far.
- The `while (link.available())` loop implements the four line-buffer steps above. `buffer[len++] = c;` stores the character and then moves the index on.
- `else if (len < MAX_LEN)` is the overflow guard. Characters beyond `MAX_LEN` are dropped, so a long message is cut short rather than overwriting memory.
- `showMessage()` prints the completed line to the Serial Monitor and the LCD.
- `lcdPrintLine()` writes exactly 16 characters, padding with spaces, so leftovers from a longer previous message are overwritten and no slow, flickering `lcd.clear()` is needed.

Task 2 exercises

2a Run it

Upload both sketches and run them. **(record)** Paste a few lines from **both** Serial Monitors. They should look like this:

Board A Serial Monitor

```
Sender ready  
Sent: Hello 0  
Sent: Test 1  
Sent: Loop 2
```

Board B Serial Monitor

```
Received: Hello 0
Received: Test 1
Received: Loop 2
```

2b Break it on purpose

Make each change below **one at a time**, observe what Board B shows, then undo it. **(record)** Fill in the table and explain each result using Section 2.

Change	What Board B shows	Why
Remove the GND-GND wire		
Change the receiver to <code>link.begin(4800)</code>		
Connect A's TX to B's TX instead of RX		
Set <code>MAX_LEN</code> to 4 on the receiver		

Note

A missing ground sometimes *appears* to work, because the boards may share a ground through the PC's USB ports. It is still wrong, and it fails as soon as one board runs from a battery.

2c Questions

1. Why does the receiver need the `'\n'` at the end of each frame? What would B display if the sender left it out?
2. What would happen if the `len < MAX_LEN` check were removed and A sent a 100-character message?

Your answer:

5 Task 3: Message integrity with SHA-1

Objective: noise, loose wires and baud-rate errors can flip bits in transit, and UART has no built-in error checking. In this task Board A attaches a **hash** of each message, and Board B recomputes the hash to decide whether the message arrived intact. The wiring is the same as Task 2, plus a push button on Board A that lets you simulate a transmission error.

Task 3: two boards + tamper button on Board A

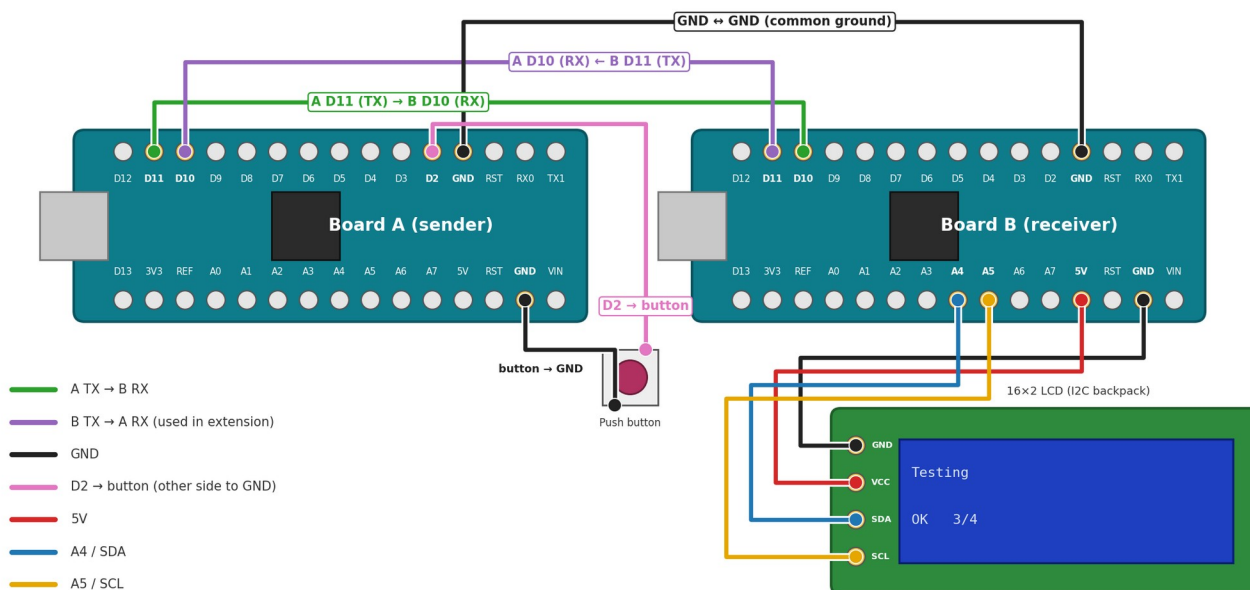


Figure 7: Task 3 wiring: the Task 2 link, plus a push button between Board A D2 and GND

Background: hash functions

A hash function turns input of any length into a **fixed-size fingerprint** called a *digest*. SHA-1 always produces 160 bits, written as **40 hexadecimal characters**. Three properties make it useful here:

- **Deterministic:** the same input always gives the same hash, so both boards will agree.
- **Avalanche effect:** changing even one bit of the input changes roughly half the output bits.
- **One-way:** you cannot work backwards from a hash to the message.

Message	SHA-1 hash
Testing	0820b32b206b7352858e8903a838ed14319acdfd
Tasting	ea1e43d415e84b553d0625a9ac75af4ea2a27fe9

One changed letter gives a completely different hash. So if the receiver's hash of the message matches the one that was sent, the message is almost certainly unchanged; if it doesn't match, something was corrupted.

Integrity is not security

SHA-1 is no longer considered secure against deliberate attacks, and a plain hash does not stop an attacker anyway: they can change the message **and** send a new matching hash. Here we only want to detect **accidental** errors. Protecting against tampering needs a secret key, for example an HMAC.

The helper library

Create a new tab in the Arduino IDE called `sha1_helper.h` (use the ▼ menu at the right of the tabs → *New Tab*) and paste in the listing from **Appendix A**. Both the Task 3 sketches, and the extension, use it through `#include "sha1_helper.h"`. You only need one function from it:

```
// Computes the SHA-1 of the C string msg and writes it as
// 40 lowercase hex characters plus '\0' into outHex
void sha1_hex(const char* msg, char outHex[41]);

// Example
char hash[41];
sha1_hex("Testing", hash);    // hash is now "0820b32b...319acdfd"
```

Memory note

The textbook SHA-1 builds an 80-word table (320 bytes) for each 64-byte block. The helper keeps only the last 16 words (64 bytes) and computes the rest on the fly. It gives the same result but saves 256 bytes of stack, over 12% of a classic Nano's RAM.

The protocol

Each message is sent as a single line: the text, a | separator, the hash, and a newline. Because the frame still ends in `'\n'`, the line-buffer receiver from Task 2 works unchanged. It just needs a bigger buffer.

Task 3 frame format

Testing		0820b32b...319acdfd	\n
message (any length)	separator	SHA-1 hash: 40 hex characters	end of frame

Board A: sender with hashing and tamper button

Task3_Sender.ino

```
// Task 3: Board A (sender with SHA-1 and tamper button)
// Extra wiring: push button between D2 and GND
#include <SoftwareSerial.h>
#include "sha1_helper.h"

SoftwareSerial link(10, 11);    // RX = D10, TX = D11

const char* const messages[] = {"Testing", "Embedded", "Software"};
const uint8_t NUM_MESSAGES = sizeof(messages) / sizeof(messages[0]);

const uint8_t BUTTON_PIN = 2;
const unsigned long SEND_INTERVAL = 2000;    // ms
const uint8_t MAX_MSG = 31;    // longest message we may tamper with

unsigned long lastSend = 0;
uint8_t msgIndex = 0;

void setup()
{
    link.begin(9600);
    Serial.begin(9600);
    pinMode(BUTTON_PIN, INPUT_PULLUP);    // reads LOW while pressed
```

```
    Serial.println(F("Sender ready"));
}

void loop()
{
    if (millis() - lastSend >= SEND_INTERVAL)
    {
        lastSend += SEND_INTERVAL;

        const char* msg = messages[msgIndex];

        // 1. Hash the ORIGINAL message
        char hexHash[41];
        sha1_hex(msg, hexHash);

        // 2. While the button is held, send a copy with one bit flipped.
        // Only copy when needed; normally the original is sent as-is.
        char corrupted[MAX_MSG + 1];
        bool tamper = (digitalRead(BUTTON_PIN) == LOW);
        if (tamper)
        {
            strncpy(corrupted, msg, MAX_MSG);
            corrupted[MAX_MSG] = '\0';
            corrupted[0] ^= 0x01;    // flip one bit, like a line error
            msg = corrupted;
        }

        // 3. Send the frame "MESSAGE|HASH\n"
        link.print(msg);
        link.print('|');
        link.print(hexHash);
        link.print('\n');

        Serial.print(tamper ? F("Sent (TAMPERED): ") : F("Sent: "));
        Serial.print(msg);
        Serial.print('|');
        Serial.println(hexHash);

        msgIndex = (msgIndex + 1) % NUM_MESSAGES;
    }
}
```

- `char hexHash[41]` holds 40 hex characters plus the `'\0'` terminator. Forgetting the `+1` is a classic buffer-overflow bug.
- The hash is always computed from the **original** message. Normally `msg` points straight at the text in `messages[]` and nothing is copied.
- **Simulating an error.** `pinMode(BUTTON_PIN, INPUT_PULLUP)` makes D2 read **HIGH** normally and **LOW** while the button is pressed. Only then does the sender copy the message into `corrupted[]` with `strncpy()`, flip one bit with `corrupted[0] ^= 0x01` (so **E** becomes **D**), and point `msg` at the copy. The original in `messages[]` is **const** and is never changed.
- No debouncing is needed: the button is only read once every 2 s, when a frame is sent.
- At 9600 baud a 50-byte frame takes about 52 ms, so the link is idle most of the time.

Board B: receiver with hash checking

Task3_Receiver.ino

```
// Task 3: Board B (receiver with SHA-1 integrity check)
#include <SoftwareSerial.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include "sha1_helper.h"

SoftwareSerial link(10, 11);          // RX = D10, TX = D11

const uint8_t LCD_COLS = 16;
LiquidCrystal_I2C lcd(0x20, LCD_COLS, 2);

const uint8_t MAX_LEN = 64;          // message + '|' + 40 hex chars fits
char buffer[MAX_LEN + 1];
uint8_t len = 0;
bool overflow = false;                // true if the current line was too long

unsigned int okCount = 0;
unsigned int failCount = 0;

// Write text on one LCD row and blank the rest of that row
void lcdPrintLine(uint8_t row, const char* text)
{
    lcd.setCursor(0, row);
    uint8_t i = 0;
    for (; i < LCD_COLS && text[i] != '\0'; i++)
    {
        lcd.write(text[i]);
    }
    for (; i < LCD_COLS; i++)
    {
        lcd.write(' ');
    }
}

// Check one complete frame of the form "MESSAGE|HASH"
void handleFrame(char* frame)
{
    // 1. Split the frame at the '|' separator
    char* sep = strchr(frame, '|');
    if (sep == NULL)
    {
        Serial.println(F("Bad frame: no '|' separator"));
        return;
    }
    *sep = '\0';                      // end the message string here
    const char* msg = frame;           // text before '|'
    const char* rxHash = sep + 1;      // text after '|'

    // 2. Hash what actually arrived
    char calcHash[41];
    sha1_hex(msg, calcHash);

    // 3. Compare with the hash the sender attached
    bool ok = (strcmp(calcHash, rxHash) == 0);
    if (ok) okCount++; else failCount++;

    Serial.print(F("Message: ")); Serial.println(msg);
    Serial.print(F("Received: ")); Serial.println(rxHash);
    Serial.print(F("Computed: ")); Serial.println(calcHash);
    Serial.println(ok ? F("Integrity: OK") : F("Integrity: FAIL"));
    Serial.println();

    char status[LCD_COLS + 1];
```

```
    snprintf(status, sizeof(status), "%s %u/%u", ok ? "OK" : "FAIL",
              okCount, okCount + failCount);
    lcdPrintLine(0, msg);
    lcdPrintLine(1, status);
}

void setup()
{
    link.begin(9600);
    Serial.begin(9600);

    lcd.init();
    lcd.backlight();
    lcdPrintLine(0, "Waiting...");
}

void loop()
{
    while (link.available())
    {
        char c = link.read();

        if (c == '\r')
        {
            continue;
        }

        if (c == '\n')
        {
            buffer[len] = '\0';
            if (overflow)
            {
                Serial.println(F("Frame too long - discarded"));
            }
            else
            {
                handleFrame(buffer);
            }
            len = 0;
            overflow = false;
        }
        else if (len < MAX_LEN)
        {
            buffer[len++] = c;
        }
        else
        {
            overflow = true;
        }
    }
}
```

- The receive loop is the Task 2 line buffer, with `MAX_LEN` raised to **64** (a 7-8 letter message + `|` + 40 hash characters, plus room to spare).
- **The overflow flag.** If a character arrives when the buffer is full, the flag is set, and when the `'\n'` arrives the whole frame is discarded. Without it, a too-long frame would be cut short and then fail the hash check for the wrong reason.
- `handleFrame()` uses `strchr()` to find the `|`, then splits the frame in place (see the box below). It hashes the message part with `sha1_hex()` and compares the result with the received hash using `strcmp()`, which returns 0 when two strings are equal.

- The result goes to the Serial Monitor in full and to the LCD in short form.
`snprintf()` builds the status line, e.g. `OK 3/4`, safely: it never writes more than `sizeof(status)` bytes.

How `strchr()` splitting works

Before: `T e s t i n g | 0 8 2 0 ... \0`

After `*sep = '\0'`: `T e s t i n g \0 0 8 2 0 ... \0`

No copying is needed. `frame` now reads as `"Testing"` and `sep + 1` reads as the 40-character hash.

Task 3 exercises

3a Run it

Add `sha1_helper.h` to both sketches, upload and run. **(record)** Paste the output from Board B's Serial Monitor. Every message should pass:

Board B Serial Monitor (one frame)

```
Message: Testing
Received: 0820b32b206b7352858e8903a838ed14319acdfd
Computed: 0820b32b206b7352858e8903a838ed14319acdfd
Integrity: OK
```

3b Simulate a transmission error

Hold the button on Board A while a message is sent. **(record)** Show the FAIL result on Board B and the matching line on Board A:

Board A Serial Monitor (button held)

```
Sent (TAMPERED): Dmbedded|0fd17b94c9576216ee0a47001e3fdbb1920a1f50
```

Board B Serial Monitor (tampered frame)

```
Message: Dmbedded
Received: 0fd17b94c9576216ee0a47001e3fdbb1920a1f50
Computed: 106b572c38133fff8835b5165f4d092b9042db0e
Integrity: FAIL
```

Only **one bit** of one character changed. How many of the 40 hex characters of the two hashes are different?

Your answer:

3c Questions

1. The message `Testing` is 7 bytes, but the frame is 49 bytes. What percentage of the link's capacity is spent on the hash? When would a smaller check, such as a checksum or CRC-8, be a better choice?

2. If the corruption hits the **hash** instead of the message, what does the receiver report? Is that the right outcome?
3. The receiver can detect errors but not fix them. How could the two boards use the second wire (B TX → A RX) so that a corrupted message gets sent again? (The extension below is one answer.)

Your answer:

6 Extension: acknowledgements (ACK/NACK)

Detecting an error is only half the job: the message still needs to get through. In this extension Board B replies to every frame on the return wire (B D11 → A D10): **ACK** if the hash matched, **NACK** if it didn't. Board A resends the same message after a **NACK**, or if no reply arrives within 500 ms, and gives up after 5 retries. This is the same idea that TCP and many radio protocols use. The wiring is the same as Task 3.

Board A: sender with retries

Extension_Sender_ACK.ino

```
// Extension: Board A (sender with SHA-1, tamper button and ACK/NACK retries)
// Same wiring as Task 3. Listens for replies on D10 (RX) from B's D11 (TX).
#include <SoftwareSerial.h>
#include "sha1_helper.h"

SoftwareSerial link(10, 11);           // RX = D10, TX = D11

const char* const messages[] = {"Testing", "Embedded", "Software"};
const uint8_t NUM_MESSAGES = sizeof(messages) / sizeof(messages[0]);

const uint8_t BUTTON_PIN = 2;
const unsigned long SEND_INTERVAL = 2000;    // ms between new messages
const unsigned long ACK_TIMEOUT = 500;       // ms to wait for a reply
const uint8_t MAX_RETRIES = 5;               // resends before giving up
const uint8_t MAX_MSG = 31;

enum State : uint8_t { IDLE, WAITING_FOR_REPLY };
State state = IDLE;

unsigned long lastSend = 0;                // when the current message was finished
unsigned long sentAt = 0;                  // when the last frame went out
uint8_t msgIndex = 0;
uint8_t retries = 0;

char reply[8];                             // "ACK" / "NACK" plus room to spare
uint8_t replyLen = 0;

// Hash and send messages[msgIndex] (corrupted while the button is held)
void sendFrame()
```

```
{
    const char* msg = messages[msgIndex];

    char hexHash[41];
    sha1_hex(msg, hexHash);

    char corrupted[MAX_MSG + 1];
    bool tamper = (digitalRead(BUTTON_PIN) == LOW);
    if (tamper)
    {
        strncpy(corrupted, msg, MAX_MSG);
        corrupted[MAX_MSG] = '\0';
        corrupted[0] ^= 0x01;           // flip one bit
        msg = corrupted;
    }

    link.print(msg);
    link.print('|');
    link.print(hexHash);
    link.print('\n');

    Serial.print(tamper ? F("Sent (TAMPERED): ") : F("Sent: "));
    Serial.println(msg);

    sentAt = millis();
    state = WAITING_FOR_REPLY;
}

// Finished with this message (delivered or given up): move on
void nextMessage()
{
    msgIndex = (msgIndex + 1) % NUM_MESSAGES;
    retries = 0;
    lastSend = millis();           // next message SEND_INTERVAL from now
    state = IDLE;
}

// Resend the same message, or give up after MAX_RETRIES
void retry()
{
    if (retries < MAX_RETRIES)
    {
        retries++;
        Serial.print(F("  Retry "));
        Serial.println(retries);
        sendFrame();
    }
    else
    {
        Serial.println(F("  Giving up on this message"));
        nextMessage();
    }
}

void handleReply(const char* r)
{
    if (state != WAITING_FOR_REPLY)
    {
        return;                     // late or unexpected reply: ignore it
    }

    if (strcmp(r, "ACK") == 0)
    {
        Serial.print(F("  ACK"));
        if (retries > 0)
        {
            Serial.print(F(" after "));
        }
    }
}
```

```
        Serial.print(retries);
        Serial.print(F(" retries"));
    }
    Serial.println();
    nextMessage();
}
else
{
    Serial.println(F(" NACK"));
    retry();
}
}

void setup()
{
    link.begin(9600);
    Serial.begin(9600);
    pinMode(BUTTON_PIN, INPUT_PULLUP);
    Serial.println(F("Sender ready"));
}

void loop()
{
    // 1. Time to send a new message?
    if (state == IDLE && millis() - lastSend >= SEND_INTERVAL)
    {
        sendFrame();
    }

    // 2. Collect reply characters into a line
    while (link.available())
    {
        char c = link.read();
        if (c == '\r')
        {
            continue;
        }
        if (c == '\n')
        {
            reply[replyLen] = '\0';
            handleReply(reply);
            replyLen = 0;
        }
        else if (replyLen < sizeof(reply) - 1)
        {
            reply[replyLen++] = c;
        }
    }

    // 3. No reply in time? Treat it like a NACK
    if (state == WAITING_FOR_REPLY && millis() - sentAt >= ACK_TIMEOUT)
    {
        Serial.println(F(" Timeout"));
        retry();
    }
}
```

- **A small state machine.** `state` is either `IDLE` (waiting until it is time to send the next message) or `WAITING_FOR_REPLY` (a frame is out and the sender is waiting for B's answer). `loop()` never blocks: it checks the time, reads any reply characters, and checks the timeout, then returns.
- `sendFrame()` is the Task 3 send code moved into a function, so it can be called both for a new message and for a retry. It records `sentAt` to start the timeout.

- Replies are collected with the same line-buffer pattern, into a small `reply[8]` buffer.
- `handleReply()`: `ACK` means the message was delivered, so `nextMessage()` moves on. `NACK` calls `retry()`, which resends the same message, or gives up after `MAX_RETRIES` so a permanent fault cannot lock the sender up forever.
- A missing reply is treated like a `NACK`. That covers the case where the frame, or the reply itself, was lost completely.
- Here `lastSend = millis()` is used on purpose: the next message is scheduled 2 s after the previous one *finished*, however many retries it took.

Board B: receiver with replies

Extension_Receiver_ACK.ino

```
// Extension: Board B (receiver with SHA-1 check and ACK/NACK replies)
// Same wiring as Task 3. Uses the return wire B D11 (TX) -> A D10 (RX).
#include <SoftwareSerial.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include "sha1_helper.h"

SoftwareSerial link(10, 11);           // RX = D10, TX = D11

const uint8_t LCD_COLS = 16;
LiquidCrystal_I2C lcd(0x20, LCD_COLS, 2);

const uint8_t MAX_LEN = 64;           // message + '|' + 40 hex chars fits
char buffer[MAX_LEN + 1];
uint8_t len = 0;
bool overflow = false;                // true if the current line was too long

unsigned int okCount = 0;
unsigned int failCount = 0;

// Write text on one LCD row and blank the rest of that row
void lcdPrintLine(uint8_t row, const char* text)
{
    lcd.setCursor(0, row);
    uint8_t i = 0;
    for (; i < LCD_COLS && text[i] != '\0'; i++)
    {
        lcd.write(text[i]);
    }
    for (; i < LCD_COLS; i++)
    {
        lcd.write(' ');
    }
}

// Tell the sender whether the frame arrived intact
void sendReply(bool ok)
{
    link.print(ok ? F("ACK\n") : F("NACK\n"));
}

// Check one complete frame of the form "MESSAGE|HASH" and reply
void handleFrame(char* frame)
{
    // 1. Split the frame at the '|' separator
    char* sep = strchr(frame, '|');
    if (sep == NULL)
    {

```

```
        Serial.println(F("Bad frame: no '|' separator"));
        sendReply(false);
        return;
    }
    *sep = '\0'; // end the message string here
    const char* msg = frame; // text before '|'
    const char* rxHash = sep + 1; // text after '|'

    // 2. Hash what actually arrived
    char calcHash[41];
    sha1_hex(msg, calcHash);

    // 3. Compare with the hash the sender attached
    bool ok = (strcmp(calcHash, rxHash) == 0);
    if (ok) okCount++; else failCount++;

    // 4. Reply straight away so the sender is not left waiting
    sendReply(ok);

    Serial.print(F("Message: ")); Serial.println(msg);
    Serial.print(F("Received: ")); Serial.println(rxHash);
    Serial.print(F("Computed: ")); Serial.println(calcHash);
    Serial.println(ok ? F("Integrity: OK") : F("Integrity: FAIL"));
    Serial.println();

    char status[LCD_COLS + 1];
    snprintf(status, sizeof(status), "%s %u/%u", ok ? "OK" : "FAIL",
             okCount, okCount + failCount);
    lcdPrintLine(0, msg);
    lcdPrintLine(1, status);
}

void setup()
{
    link.begin(9600);
    Serial.begin(9600);

    lcd.init();
    lcd.backlight();
    lcdPrintLine(0, "Waiting...");
}

void loop()
{
    while (link.available())
    {
        char c = link.read();

        if (c == '\r')
        {
            continue;
        }

        if (c == '\n')
        {
            buffer[len] = '\0';
            if (overflow)
            {
                Serial.println(F("Frame too long - discarded"));
                sendReply(false);
            }
            else
            {
                handleFrame(buffer);
            }
            len = 0;
            overflow = false;
        }
    }
}
```

```
    }  
    else if (len < MAX_LEN)  
    {  
        buffer[len++] = c;  
    }  
    else  
    {  
        overflow = true;  
    }  
}  
}
```

This is the Task 3 receiver with one addition: `sendReply()` sends `ACK\n` or `NACK\n` back to A. It is called for every frame, including bad or too-long frames, so the sender is never left waiting for a reply that will not come.

Why SoftwareSerial still works here

A SoftwareSerial port cannot receive while it is transmitting. That is fine here because the two boards take turns: A only listens after it has finished sending a frame, and B only replies after it has received the whole frame.

Extension exercises

E1 Run it

Upload both sketches. **(record)** Tap the button briefly so that one or two frames are corrupted, and paste Board A's output. You should see a retry succeed:

Board A Serial Monitor

```
Sent: Testing  
ACK  
Sent (TAMPERED): Dmbedded  
NACK  
Retry 1  
Sent: Embedded  
ACK after 1 retries
```

E2 Hold and unplug

(record) (a) Hold the button down for several seconds. What does Board A do, and why is giving up better than retrying forever? (b) Pull out the B TX → A RX wire for a few seconds. What does A report now, and what happens when you reconnect it?

Your answer:

7 Submission checklist

- Task 1: LCD photo and answers to 1b, 1c and 1d.
- Task 2: Serial Monitor output from both boards, the completed “break it” table, and answers to 2c.
- Task 3: an OK and a FAIL output from both boards, and answers to 3b and 3c.
- Extension (optional): Board A output showing a successful retry, and answers to E2.

Appendix A: sha1_helper.h

Paste this into a new IDE tab named exactly `sha1_helper.h`, in every sketch that uses it (Task 3 and the extension, both boards). You do not need to understand the internals, but the comments show the main steps.

sha1_helper.h

```
#ifndef SHA1_HELPER_H
#define SHA1_HELPER_H

// -----
// Minimal SHA-1 for Arduino (classic Nano and Nano Every)
//
// Usage:
//   char hash[41];
//   sha1_hex("Testing", hash);   // hash = "0820b32b...319acdfd"
//
// RAM-friendly: the message schedule uses a 16-word rolling
// buffer (64 bytes) instead of the textbook 80-word array
// (320 bytes), which matters on a 2 KB ATmega328P.
// -----

#include <Arduino.h>
#include <stdint.h>
#include <string.h>

class SimpleSHA1 {
public:
    SimpleSHA1() { reset(); }

    void reset() {
        _dlen = 0; _bits = 0;
        _st[0] = 0x67452301UL; _st[1] = 0xEFCDAB89UL; _st[2] = 0x98BADCFEUL;
        _st[3] = 0x10325476UL; _st[4] = 0xC3D2E1F0UL;
    }

    void update(const uint8_t* data, size_t len) {
        for (size_t i = 0; i < len; i++) {
            _blk[_dlen++] = data[i];
            if (_dlen == 64) { transform(); _bits += 512; _dlen = 0; }
        }
    }

    void finalize(uint8_t out[20]) {
        _bits += (uint64_t)_dlen * 8ULL;
        uint8_t i = _dlen;
        _blk[i++] = 0x80;
        if (i > 56) {
            // no room for the length:
            // pad this block, start another
            while (i < 64) _blk[i++] = 0x00;
            transform();
            i = 0;
        }
        while (i < 56) _blk[i++] = 0x00;
        for (uint8_t j = 0; j < 8; j++) {
            // 64-bit length, big-endian
            _blk[63 - j] = (uint8_t)(_bits >> (8 * j));
        }
        transform();
        for (uint8_t j = 0; j < 5; j++) {
            out[j * 4 + 0] = (uint8_t)(_st[j] >> 24);
            out[j * 4 + 1] = (uint8_t)(_st[j] >> 16);
            out[j * 4 + 2] = (uint8_t)(_st[j] >> 8);
            out[j * 4 + 3] = (uint8_t)(_st[j]);
        }
    }
}
```

```
private:
    uint8_t _blk[64];
    uint8_t _dlen;
    uint64_t _bits;
    uint32_t _st[5];

    static inline uint32_t rol(uint32_t x, uint8_t n) { return (x << n) | (x >> (32 - n)); }

void transform() {
    uint32_t w[16]; // rolling schedule: w[t & 15]
    for (uint8_t i = 0; i < 16; i++) {
        w[i] = ((uint32_t)_blk[i * 4] << 24) | ((uint32_t)_blk[i * 4 + 1] << 16) |
            ((uint32_t)_blk[i * 4 + 2] << 8) | _blk[i * 4 + 3];
    }
    uint32_t a = _st[0], b = _st[1], c = _st[2], d = _st[3], e = _st[4];
    for (uint8_t t = 0; t < 80; t++) {
        if (t >= 16) {
            w[t & 15] = rol(w[(t + 13) & 15] ^ w[(t + 8) & 15] ^
                w[(t + 2) & 15] ^ w[t & 15], 1);
        }
        uint32_t f, k;
        if (t < 20) { f = (b & c) | (~b & d); k = 0x5A827999UL; }
        else if (t < 40) { f = b ^ c ^ d; k = 0x6ED9EBA1UL; }
        else if (t < 60) { f = (b & c) | (b & d) | (c & d); k = 0x8F1BBCDCUL; }
        else { f = b ^ c ^ d; k = 0xCA62C1D6UL; }
        uint32_t tmp = rol(a, 5) + f + e + k + w[t & 15];
        e = d; d = c; c = rol(b, 30); b = a; a = tmp;
    }
    _st[0] += a; _st[1] += b; _st[2] += c; _st[3] += d; _st[4] += e;
}

};

// Lowercase hex SHA-1 of a C string into outHex[41] (40 chars + '\0')
inline void sha1_hex(const char* msg, char outHex[41]) {
    static const char HEXCHARS[] = "0123456789abcdef";
    SimpleSHA1 s;
    uint8_t d[20];
    s.update((const uint8_t*)msg, strlen(msg));
    s.finalize(d);
    for (uint8_t i = 0; i < 20; i++) {
        outHex[i * 2] = HEXCHARS[d[i] >> 4];
        outHex[i * 2 + 1] = HEXCHARS[d[i] & 0x0F];
    }
    outHex[40] = '\0';
}

#endif // SHA1_HELPER_H
```